

Progetto Computer Architecture- Network Interface

Margherita Rossi

Luglio 2026

Repository: <https://github.com/mrossi15/vhdl-network-interface>

1 Introduzione

L'obiettivo di questo progetto consiste nell'implementazione e simulazione VHDL di un'interfaccia di rete hardware (Network Interface) dedicata alla gestione, serializzazione e instradamento di flussi dati strutturati a pacchetti su parole a 32-bit (*word*).

Il sistema adotta un **protocollo a pacchetti**, in cui il flusso informativo continuo viene suddiviso in unità discrete composte da due sezioni fondamentali:

- **Header:** contiene le informazioni di controllo necessarie alla gestione del pacchetto (lunghezza del messaggio e campi di indirizzamento della destinazione, definiti nel pacchetto `performance_type_pkg`).
- **Payload:** rappresenta il contenuto informativo effettivo che si desidera trasferire

Dal punto di vista architetturale, il sistema è suddiviso in due blocchi funzionali cooperanti:

1. **Blocco di Trasmissione (TX BLOCK)** Fig. 1: Si occupa dell'acquisizione dal generatore (attraverso FSM1), del posizionamento in un buffer locale (FIFO) e della serializzazione del pacchetto su un canale a singola linea (attraverso FSM2).
2. **Blocco di Ricezione (RX BLOCK)** Fig. 9: Effettua la deserializzazione del flusso seriale, la decodifica dell'header e l'instradamento del dato verso la FIFO di destinazione corretta (attraverso FSM3).

La validazione del design è stata condotta secondo un approccio incrementale. Il modulo principale PROJECT è stato infine integrato in un modulo di alto livello denominato TOP. Infine è stata implementata la struttura su una FPGA reale, modello Xilinx VCU118 e ne è stato verificato il funzionamento.

2 Blocco di Trasmissione

Per ciascuno dei moduli descritti in questa sezione è stato predisposto un Test Bench dedicato. La metodologia di verifica adottata è comune a tutti i blocchi e si concentra su quattro aspetti: l'integrità dei dati lungo il percorso (assenza di corruzioni tra ingresso e uscita), il corretto funzionamento del meccanismo di flow control (freeze/ripresa in caso di buffer pieno o assenza di dati), la coerenza tra i segnali di controllo (validità del dato e abilitazione alla scrittura/lettura) e la correttezza delle transizioni di stato della macchina. Nei paragrafi seguenti si riportano, per ogni modulo, solo le osservazioni specifiche emerse dalle rispettive simulazioni.

2.1 Packet Generator

Per implementare correttamente il protocollo è stato necessario analizzare inizialmente il *Packet Generator* a disposizione. Questo infatti inizializza la creazione dei dati che vengono inseriti all'inizio del percorso in base ai dati contenuti nel modulo `performance_type_pkg`.

Il trasferimento dei dati verso il modulo successivo avviene attraverso un protocollo di hand-shake sincrono basato su due segnali fondamentali. Uno schema dei segnali in entrata e uscita del Packet Generator sono riportati in Fig. 1. Il trasferimento di una *word* avviene con successo solo sul fronte positivo del clock in cui entrambi i segnali `dt_valid` e `dt_ready` sono contemporaneamente alti ('1'). Se il modulo a valle abbassa il segnale `dt_ready` (ad esempio perché la FIFO è piena), il Packet Generator congela la trasmissione, mantenendo il dato corrente e il segnale `dt_valid` alti fino alla ripresa del flusso. Per verificare il corretto comportamento del modulo e la sua conformità al protocollo Valid/Ready, è stato predisposto un apposito TestBench.

Le Fig. 2 e 3 mostrano le forme d'onda della simulazione. Applicando la metodologia di verifica descritta in precedenza, si osserva in particolare che il bus dati presenta l'Header per un singolo ciclo di clock prima di

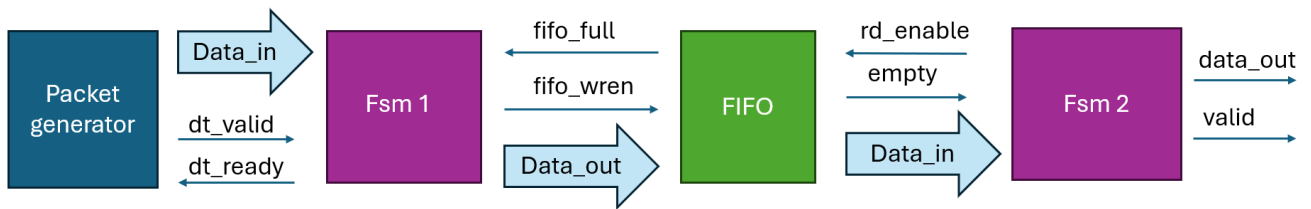


Figura 1: Schema del blocco di Trasmissione e Packet Generator

passare sequenzialmente alle word del Payload , con corretta attivazione di `h_dt_wren`, `w_dt_wren` e `dt_valid` (Fig.2). Il meccanismo di handshake risulta inoltre robusto: abbassando `dt_ready`, il Packet Generator congela istantaneamente l'evoluzione del dato mantenendo `dt_valid` alto, per poi riprendere la trasmissione esattamente dal punto di sospensione non appena `dt_ready` torna a '1', senza perdita di informazioni (Fig.3).

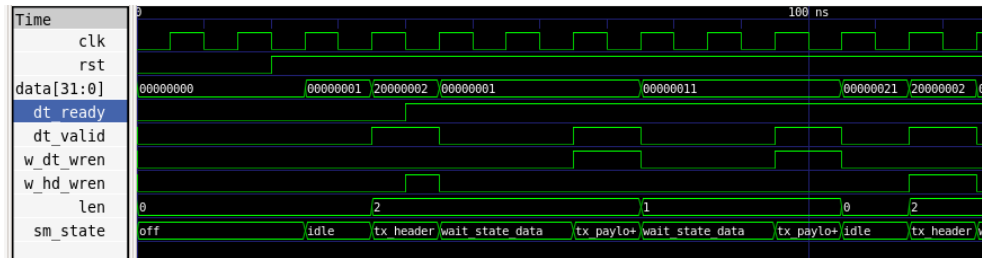


Figura 2: Forme d'onda della simulazione del Packet Generator.

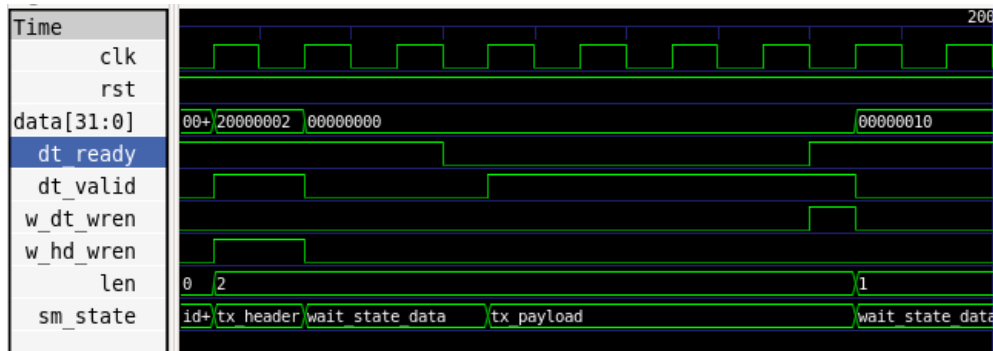


Figura 3: Forme d'onda della simulazione del Packet Generator: comportamento dell'handshake in presenza di `dt_ready` basso, con congelamento e ripresa della trasmissione senza perdita di dati

2.2 FSM 1

La macchina a stati FSM 1 si occupa di permettere la comunicazione tra il Packet Generator e la FIFO (FWFT). Le porte del componente sono riportate in Fig. 1 insieme alle altre componenti del blocco di Trasmissione.

La macchina a stati è stata implementata seguendo due processi (uno puramente sequenziale sincronizzato sul fronte di salita del clock e uno combinatorio per la logica di transizione e delle uscite). La FSM si compone di due stati principali, descritti nel diagramma logico di Fig. 4:

- **IDLE:** È lo stato di riposo in cui la macchina attende l'arrivo di un pacchetto. Se la FIFO non è piena (`fifo_full` = '0'), la FSM asserisce `dt_ready` <= '1' verso il generatore. Non appena viene rilevato un dato valido (`dt_valid` = '1'), la FSM abilita la scrittura in FIFO (`fifo_wren` <= '1') per catturare l'Header. Contemporaneamente, decodifica il campo della lunghezza del Payload dall'Header stesso:
 - Se la lunghezza del Payload è pari a zero, la macchina rimane in **IDLE**.
 - Se il Payload contiene dei dati, il contatore interno (`contatore_reg`) viene caricato con il numero di *word* da ricevere e la macchina passa allo stato di **PAYLOAD**.
- **PAYLOAD:** In questo stato la macchina acquisisce sequenzialmente le *word* di dati. Ad ogni ciclo di clock in cui l'handshake va a buon fine (`dt_valid` = '1' e la FIFO ha spazio), il dato viene scritto nel buffer e

il contatore interno viene decrementato di un'unità. Quando il contatore raggiunge il valore 1, significa che l'ultima *word* del Payload è stata elaborata; pertanto, la FSM ripristina il contatore a zero e ritorna nello stato **IDLE**.

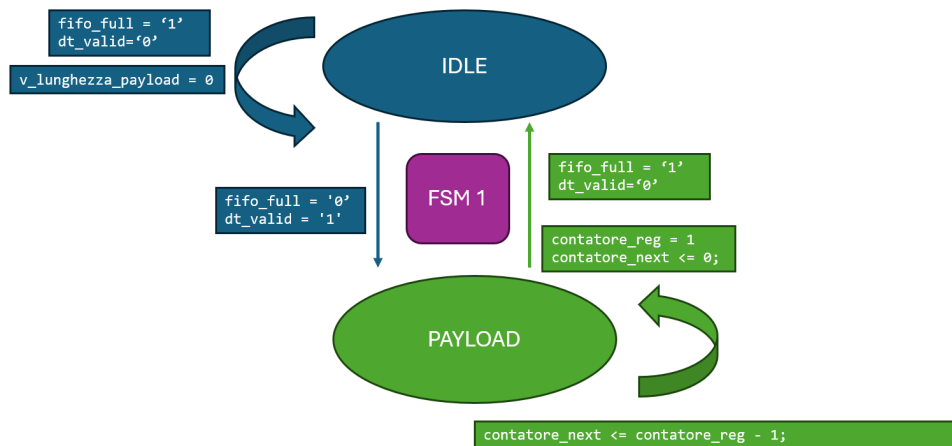


Figura 4: Schema Logico FSM1

Il Test Bench predisposto per FSM1 integra Packet Generator, FSM1 e FIFO. Applicando la metodologia generale di verifica, i risultati (Fig. 5 e 6) confermano che il bus dati in uscita verso il buffer replica fedelmente l'ingresso, senza corruzioni di Header o Payload. In particolare, a differenza del Packet Generator, qui il segnale di flow control osservato è `fifo_full`: non appena asserito, la FSM abbassa istantaneamente `dt_ready`, riprendendo la ricezione dal punto esatto di interruzione una volta liberato spazio in FIFO (Fig. 6). La scrittura in FIFO (`fifo_wren`) risulta correttamente abilitata solo in presenza congiunta di dato valido e spazio disponibile, e le transizioni IDLE, PAYLOAD avvengono nei tempi previsti.

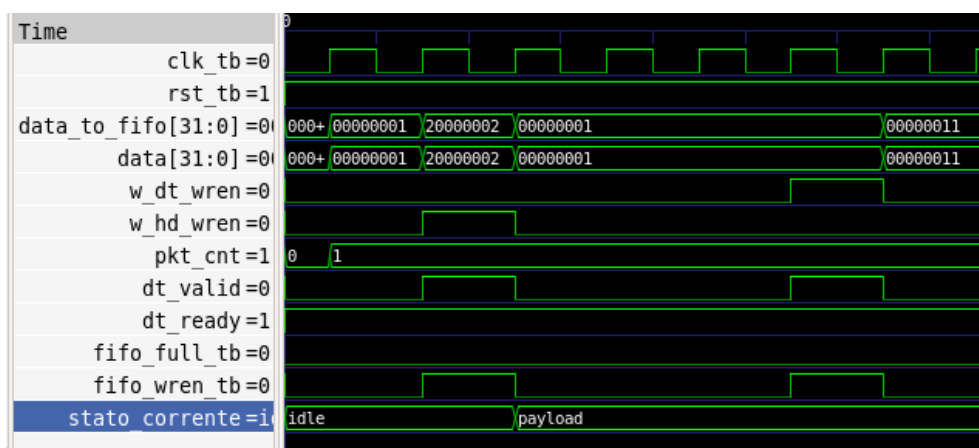


Figura 5: Forme d'onda di FSM1: transizione IDLE→PAYLOAD e corretta scrittura in FIFO in condizioni nominali (`fifo_full = '0'`)

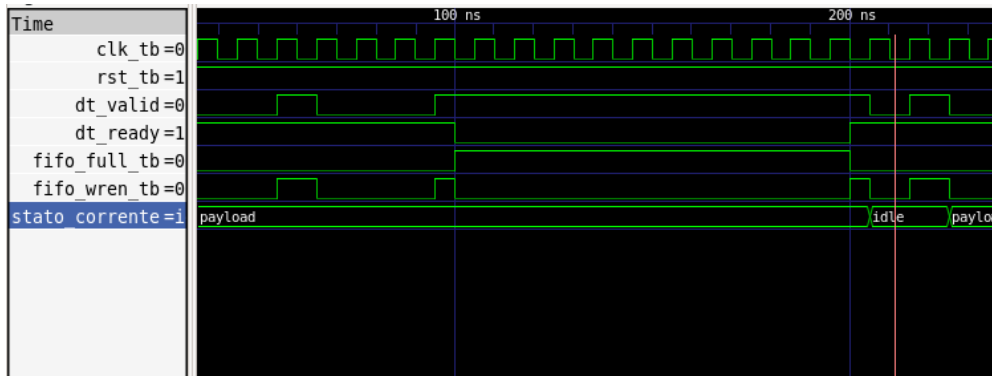


Figura 6: Forme d'onda della simulazione FSM 1, in particolare è possibile notare il corretto andamento dei segnali nel caso in cui la fifo sia piena (`fifo_full_tb`)

2.3 FSM2

La macchina a stati **FSM2** ha come obiettivo la serializzazione dei dati in ingresso provenienti dalla FIFO. Per l'architettura di memorizzazione è stata scelta una FIFO con meccanismo *First-Word-Fall-Through* (FWFT). Le porte logiche del componente sono illustrate in Fig.1.

La FSM è strutturata in quattro stati principali per separare nettamente la gestione dell'header da quella del payload. Durante gli stati di lettura (**READ**), viene abilitato il segnale di lettura della FIFO e il dato in uscita viene contestualmente memorizzato in un registro locale, garantendone la stabilità per tutte le fasi successive. Una volta acquisito il dato, la macchina evolve verso il rispettivo stato di serializzazione (**SERIALIZE**), dove la *word* viene trasmessa in uscita bit a bit sfruttando un contatore (`bit_counter`) che decrementa ad ogni ciclo di clock.

- **READ HEADER:** stato iniziale di reset. Al rilevamento di dati nella FIFO (modalità FWFT), il sistema campiona l'header in `header_reg`, attiva `fifo_rd_en` e inizializza il `bit_counter_reg`, per poi passare alla serializzazione.
- **SERIALIZE HEADER:** trasmette l'header serialmente (da MSB a LSB) mantenendo `valid_out` alto. Terminato l'invio dei 32 bit, la logica decodifica il campo lunghezza: se è presente un payload, carica il `word_counter` ed evolve in **READ PAYLOAD**; altrimenti, torna in **READ HEADER** in attesa di nuovi pacchetti.
- **READ PAYLOAD:** acquisisce sequenzialmente le *word* del payload. A ogni ciclo, verificata la disponibilità del dato, aggiorna `payload_reg`, decrementa il `word_counter` e prepara il `bit_counter` per la trasmissione, transitando poi nello stato di serializzazione.
- **SERIALIZE PAYLOAD:** trasmette serialmente (da MSB a LSB) il contenuto di `payload_reg` con `valid_out` attivo, decrementando progressivamente il `bit_counter`. Terminato l'invio, valuta il `word_counter`: se vi sono altre *word* da trasmettere, la FSM torna in **READ PAYLOAD**; altrimenti, il pacchetto è completo e la macchina si riporta allo stato iniziale **READ HEADER**.

Il Test Bench per FSM2 estende la catena di verifica a Packet Generator, FSM1, FIFO e FSM2 (Fig. 7, 8). Oltre alla corretta sequenza di stati (**READ/SERIALIZE HEADER** → **READ/SERIALIZE PAYLOAD** → ritorno stabile allo stato iniziale), le simulazioni evidenziano due aspetti specifici di questo modulo: l'efficacia del meccanismo FWFT della FIFO, per cui il dato è già disponibile sul bus nello stesso ciclo in cui viene asserito `fifo_rd_en`, senza cicli di latenza; e la corretta generazione di `valid_out`, che si porta a '1' in concomitanza con il primo bit trasmesso e resta asserito per tutta la serializzazione. A livello di bit, il `bit_counter` decrementa regolarmente da 31 a 0 ad ogni fronte di clock, e il confronto tra word parallela in ingresso e flusso seriale in uscita conferma l'assenza di perdite o alterazioni (Fig. 8).

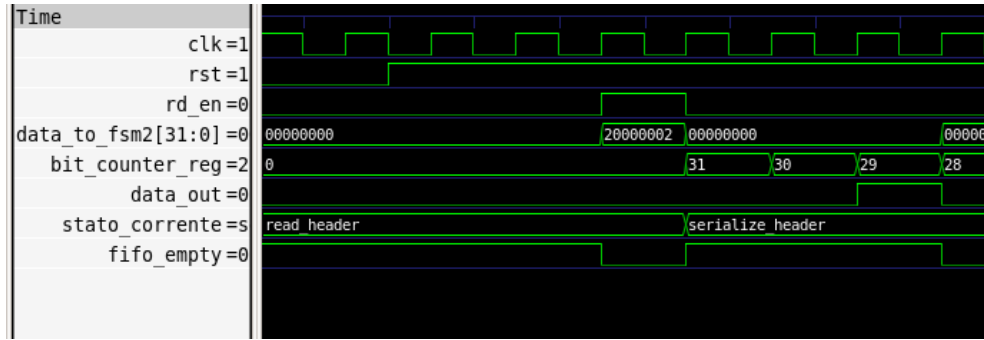


Figura 7: Forme d'onda di FSM2: sequenza READ_HEADER→ SERIALIZE_HEADER e campionamento dell'header in bit_counter_reg

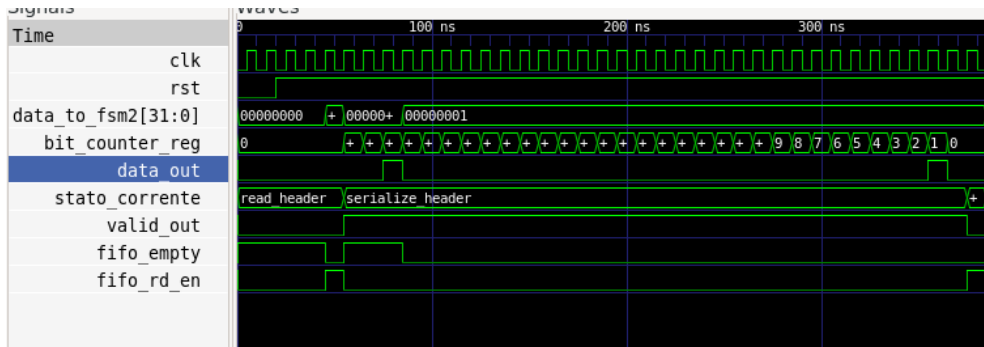


Figura 8: Forme d'onda della simulazione della FSM2, dettaglio sui valori in uscita, si noti come il valore in uscita dalla FIFO (001000...0010) sia correttamente serializzato dalla FSM2

3 Blocco di Ricezione

3.1 FSM3

Con la terza macchina a stati inizia il blocco di ricezione. Uno schema che lo descriva è riportato in Fig. 9.

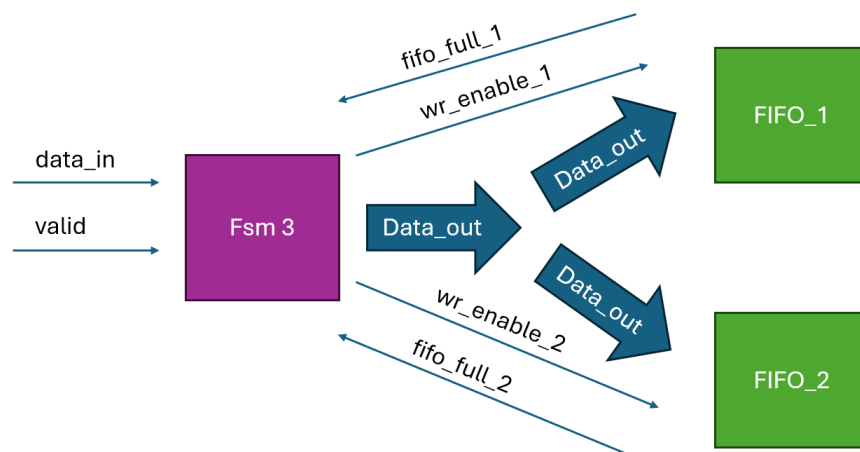


Figura 9: Schema del blocco di Ricezione

Per realizzare la FSM3 senza inserire un segnale di controllo di flusso (come un ready) in direzione della FSM2, la struttura della macchina a stati deve seguire uno schema ottimizzato per l'acquisizione continua dei dati: il segnale seriale in ingresso non può mai essere ignorato, pena la perdita di sincronizzazione con il trasmettitore. Il design si articola in due soli stati operativi, descritti nel diagramma logico di Fig. 10:

- **READ_HEADER:** stato iniziale di reset. Ad ogni ciclo di clock in cui `valid = '1'`, il bit corrente presente su `data_in` viene deserializzato e inserito nella posizione indicata da `bit_counter` all'interno di una variabile

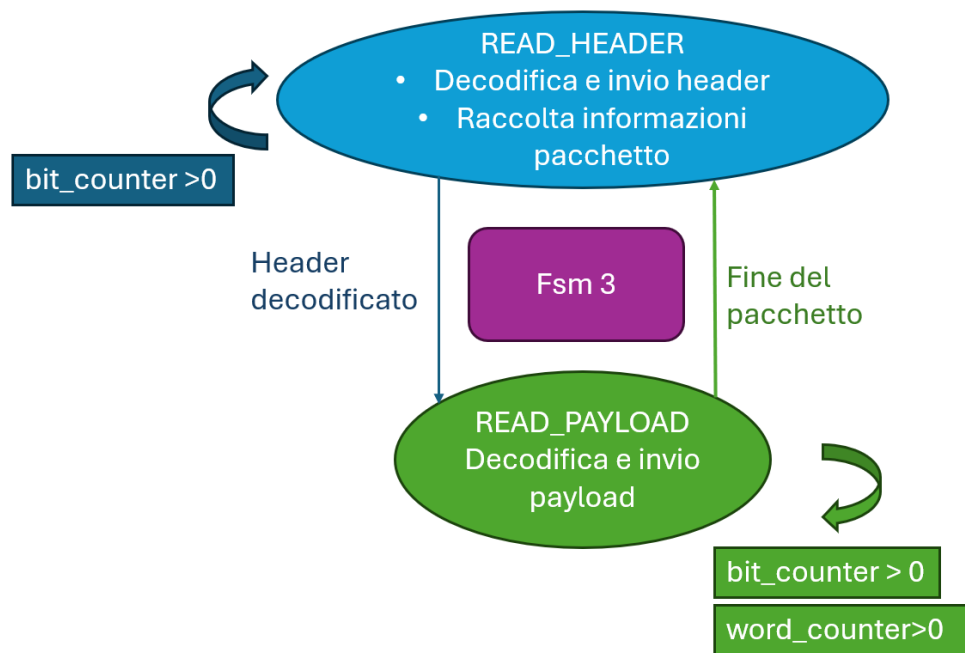


Figura 10: Schema Logico FSM3

locale (`v_header`), a partire dal MSB (bit 31) fino al LSB. Il contatore decrementa a ogni bit ricevuto. Quando `bit_counter` raggiunge 0, l'Header è completo: in questo stesso ciclo di clock, senza attendere la transizione allo stato successivo, la FSM decodifica il campo di lunghezza del Payload (caricando `word_counter`) e il campo di indirizzamento della FIFO di destinazione. In base al valore letto, viene selezionata `dest_fifo_reg` ("10" per FIFO_1, "01" per FIFO_2, "00" in caso di indirizzo non valido) e, se la FIFO corrispondente non è piena, viene subito asserito il relativo `wr_enable` per scrivere l'Header appena ricostruito. L'uso di variabili locali (anziché dei soli segnali registrati) consente di decodificare e scrivere l'Header nello stesso ciclo di clock in cui ne viene ricevuto l'ultimo bit. Questa scelta rende la FSM indipendente dal comportamento temporale di FSM2: introdurre uno stato di sola decodifica avrebbe funzionato solo grazie alla pausa di un ciclo che FSM2 introduce naturalmente tra la serializzazione dell'Header e quella del Payload (durante la lettura della word successiva dalla FIFO), un'assunzione che potrebbe venire meno se il trasmettitore fosse in futuro modificato o ottimizzato.

- **READ_PAYLOAD:** la macchina ricostruisce in modo analogo, bit a bit, le word del Payload in una variabile locale (`v_payload`). Al completamento di ciascuna word (`bit_counter = 0`), il dato viene scritto nella FIFO indicata da `dest_fifo_reg` (con lo stesso controllo sul relativo segnale di full) e il `word_counter` viene decrementato. Quando quest'ultimo raggiunge il valore 1, significa che l'ultima word del pacchetto è stata elaborata: la FSM ripristina `dest_fifo_reg` e ritorna in READ HEADER, pronta ad acquisire un nuovo pacchetto; altrimenti resta in READ PAYLOAD per la word successiva.

Per validare il corretto funzionamento della FSM3 è stato predisposto un Test Bench dedicato. Oltre alla metodologia generale di verifica descritta per il blocco di trasmissione, il testbench si concentra su un aspetto critico proprio di questo modulo: l'assenza di un segnale di ready verso FSM2 richiede che il flusso seriale in ingresso venga correttamente campionato o correttamente ignorato (quando `valid = '0'`) in ogni ciclo di clock, senza margine di errore nella gestione dei bit. Le simulazioni (Fig. 11) confermano inoltre:

- La correttezza del routing verso la FIFO indicata dal campo di indirizzamento dell'Header, salvato in `dest_fifo_reg` e mantenuto stabile per tutta la ricezione del Payload;
- L'integrità di `header_reg` e `payload_reg`, il cui contenuto viene campionato correttamente tramite le variabili locali e scritto in uscita nello stesso ciclo di clock in cui si completa la ricezione dell'ultimo bit, senza introdurre stati di attesa aggiuntivi;
- La corretta subordinazione della scrittura in FIFO al relativo segnale di full: il Test Bench ha verificato che, quando la FIFO di destinazione non presenta spazio disponibile, il segnale di `wr_enable` non venga asserito, evidenziando il limite architetturale legato all'assenza di un meccanismo di ritrasmissione in tali condizioni.

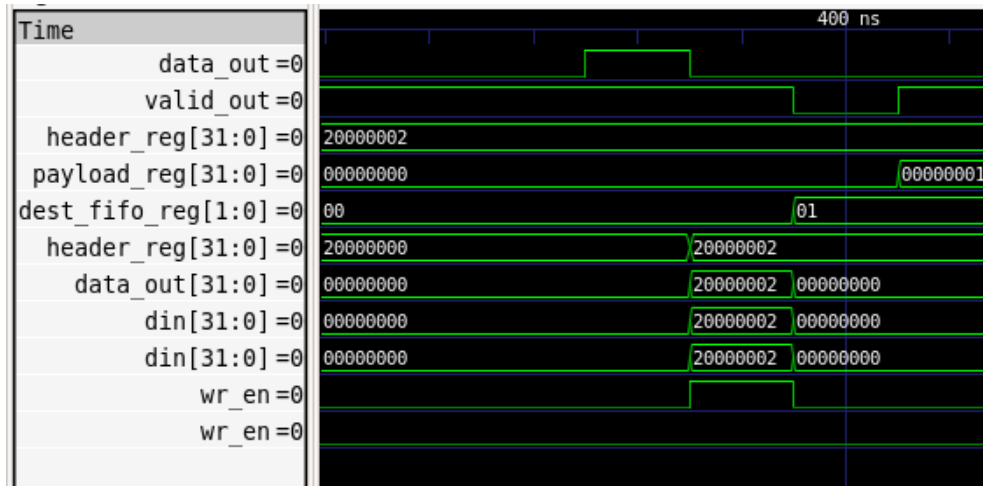


Figura 11: Forme d'onda della simulazione FSM3, in particolare si noti come nel registro la destinazione sia salvata correttamente e come i dati arrivino in entrambe le entrate della FIFO ma solo una delle due mostra il `wr_enable` alto

4 Top Level

L'integrazione a livello di sistema (*Top Level*) connette l'intera catena di elaborazione: dalla generazione parallela dei dati, alla loro serializzazione per il trasferimento su singolo bit, fino alla ricezione, deserializzazione e instradamento finale. La struttura del modulo TOP collega i blocchi principali come segue:

- **tx_block**: racchiude la logica di acquisizione, la FIFO di bufferizzazione locale e la macchina di serializzazione (FSM2) per l'invio sul canale seriale.
- **rx_block**: acquisisce il flusso seriale, effettua la deserializzazione tramite FSM3, decodifica l'indirizzo dal campo Header (bit 31..29) e smista il Payload verso una delle due FIFO di destinazione.
- **packet_checker**: due istanze di questo modulo leggono i dati ricostruiti dalle FIFO di destinazione e li confrontano direttamente con i pacchetti originariamente emessi dal **packet_generator**.

Il corretto funzionamento dell'intera catena e l'integrità dei dati sono confermati dall'asserzione dei flag di validazione `test_ok_1` e `test_ok_2`

5 Implementazione su FPGA

La fase finale del progetto ha previsto l'implementazione del sistema su un dispositivo FPGA fisico. Il lavoro è stato svolto presso i laboratori APE dell'Università Sapienza, impiegando l'ambiente di sviluppo Xilinx Vivado e una scheda FPGA modello Xilinx VCU118.

Il design è stato realizzato sfruttando le *IP core* messe a disposizione dal tool, in particolare per la generazione della memoria FIFO e per la gestione del clock (tramite il *Clocking Wizard*). Nello specifico, per la gestione delle frequenze è stato utilizzato un clock in ingresso di 300 MHz per generare un clock in uscita di 100 MHz, adottando un'architettura con domini di clock asincroni.

In una prima fase, l'esecuzione di una simulazione tramite testbench ha permesso di validare il progetto, replicando con successo i risultati già verificati in precedenza tramite il simulatore GHDL. Successivamente, il comportamento del sistema è stato validato *on-chip* monitorando i segnali attraverso l'inserimento di un modulo ILA (*Integrated Logic Analyzer*). L'analisi del timing post-implementazione condotta in Vivado ha confermato la chiusura positiva dei vincoli temporali per il clock a 100 MHz, con uno slack positivo sul percorso critico (9 ns), a conferma che la frequenza operativa scelta risulta ampiamente sostenibile dall'architettura implementata. Di seguito sono riportate le immagini acquisite tramite l'Hardware Manager di Vivado, le quali illustrano le risorse attive della FPGA e i pin esterni impiegati, nello specifico: `clk_p`, `clk_n`, `reset`, `test_ok_1` e `test_ok_2`.

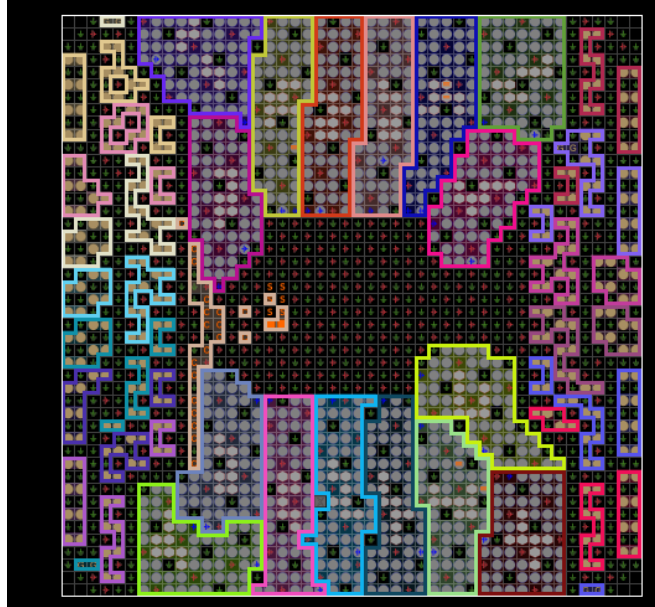


Figura 12: Posizione dei pin esterni, in particolare i pin impegnati si può vedere : `clk_p`, `clk_n`, `reset`, `test_ok.1` e `test_ok.2`

6 Conclusioni

Il progetto ha permesso di realizzare e validare un'interfaccia di rete completa, capace di gestire l'intero ciclo di vita di un pacchetto dati: dalla generazione parallela, attraverso la bufferizzazione e la serializzazione (blocco TX), fino alla ricezione, deserializzazione e instradamento verso la destinazione corretta (blocco RX). L'approccio incrementale adottato — validazione separata di ciascuna FSM tramite testbench dedicati, seguita dall'integrazione nel modulo TOP — si è rivelato efficace nell'isolare tempestivamente eventuali malfunzionamenti, in particolare nella gestione del protocollo di handshake Valid/Ready (FSM1) e nella sincronizzazione priva di flow control tra FSM2 e FSM3. Le forme d'onda ottenute confermano la persistenza dei dati, la corretta coordinazione tra i segnali di controllo interni e l'assenza di perdita o duplicazione di informazioni in ogni scenario testato, incluse le condizioni di stallo dovute al riempimento della FIFO. Tra i possibili sviluppi futuri del progetto si segnalano: l'introduzione di un meccanismo di rilevazione o correzione d'errore sul flusso seriale, attualmente assunto privo di corruzioni; l'estensione del numero di FIFO di destinazione oltre le due attualmente previste, generalizzando la logica di routing di FSM3.

Il codice sorgente VHDL e i testbench discussi in questo documento sono disponibili pubblicamente al repository: <https://github.com/mrossi15/vhdl-network-interface>



Figura 13: Risorse attive su FPGA (VCU118) per l'implementazione del progetto: distribuzione dei blocchi logici utilizzati sui vari Super Logic Region (SLR)